COURSE SYLLABUS

Introduction to Programming Using Python

COURSE NUMBER	DURATION	AUDIENCE LEVEL	FORMAT
55264B	5 days · 16 modules	Beginner	Instructor-Led / Self-Paced

Course syllabus

COURSE OVERVIEW

This five-day instructor-led course is designed for students who want to master Python programming with a heavy emphasis on Artificial Intelligence (AI) and Machine Learning (ML) infrastructure. Grounded in Python 3.12, this curriculum teaches students how to write, debug, and document clean code while constructing data pipelines, processing mathematical vectors, and integrating live cloud-based Machine Learning models. Lab exercises are conducted within Google Colab, providing zero-configuration access to GPU runtimes, and students will learn to deploy production-grade Python interfaces to free cloud-hosting platforms directly from GitHub repositories.

AT COURSE COMPLETION

After completing this course, you will be able to:

Implement robust Python 3.12 data structures, type annotations, and mathematical operations optimized for machine learning feature arrays and tensor inputs.

Construct high-performance data engineering pipelines using loops, comprehensions, generators, and functional programming patterns across cloud-based Google Colab runtimes.

Consume live data streaming patterns by building custom Web API integrations, JSON serialization pipes, and authenticated LLM API connections.

Build enterprise-grade, highly testable AI programs by leveraging strict exception handling, custom exceptions, structured logging, and PEP 8-compliant code standards.

Develop and publicly deploy interactive Python AI web applications using Streamlit, GitHub version control, and free cloud hosting platforms.

PREREQUISITES

- Experience performing basic command-line operations on Windows, Linux, or Mac computers.

- Six months of experience writing code in any programming language is recommended.

COURSE OUTLINE

MODULE 1 Variables, Type Hinting, and Mathematical Operators

150 min

This module combines fundamental data storage paradigms with algorithmic mathematical operations required to evaluate numerical formulas. Students learn to declare variables, apply static type annotations, and execute arithmetic and logical operations using Python 3.12 syntax.

LESSONS

- **Lesson 1:** Dynamic Variable Assignment and Core Data Types in Python 3.12
- **Lesson 2:** Static Type Hinting and Variable Annotations for Feature Safety
- **Lesson 3:** Arithmetic Operators, Exponentiation, and Modulus Evaluations
- **Lesson 4:** Logical and Comparison Operators for Conditional Data Validation
- **Lesson 5:** The Walrus Operator (:=) and Assignment Expressions
- **Lesson 6:** Navigating the Google Colab Runtime for Python Development

LAB EXERCISES

Variables, Type Hinting, and Mathematical Operators in Google Colab

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

Assign appropriate data types to variables dynamically inside a Google Colab execution environment.

Apply static type hinting and variable annotations to safeguard multidimensional feature properties.

Execute arithmetic operations, modulus remainder evaluations, and logical comparison matrices.

Utilize the walrus operator (:=) to streamline variable tracking inside expressions.

Construct an annotated data-vector template and mathematical formula evaluator in Google Colab.

MODULE 2 **Data Collections for Feature Engineering** 150 min

This module explores compound data structures used to catalog, manipulate, and sort large batches of training datasets. Students build lists, tuples, dictionaries, and sets, learning how each structure supports different aspects of ML feature management.

LESSONS

- **Lesson 1:** Lists: Mutable Sequences for Dynamic Feature Array Tracking
- **Lesson 2:** Tuples: Immutable Records for Static Label Storage
- **Lesson 3:** Dictionaries: Key-Value Mapping for Metadata and Feature Schemas
- **Lesson 4:** Advanced Dictionary Techniques: Merging, Updating, and Comprehension Patterns
- **Lesson 5:** Sets: Deduplication Engines for Cleaning Data Streams
- **Lesson 6:** Selecting the Right Collection Type for Machine Learning Workloads

LAB EXERCISES

- Building a Feature Engineering Toolkit with Python Collections

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

- Create and manipulate mutable lists for tracking array targets and immutable tuples for static labels.
- Map unstructured metadata using key-value structures inside dictionaries.
- Leverage sets for deduplicating data streams and computing mathematical member intersections.
- Select the appropriate collection type based on mutability, ordering, and uniqueness requirements.
- Construct a multi-tier training data dictionary and feature indexer in Google Colab.

MODULE 3 Standard Library Utilities for Data Workflows

142 min

This module explains how to organize and run core operations using native Python environments without relying on external codebases. Students import modules using aliases, extract environment metadata with sys, and manage cross-platform file paths with pathlib.

LESSONS

- **Lesson 1:** Module Import Patterns: Aliases, Selective Imports, and Namespace Safety
- **Lesson 2:** Exploring Modules Interactively with help() and dir()
- **Lesson 3:** The sys Module: Extracting Runtime Metadata and Environment Paths
- **Lesson 4:** The pathlib Framework: Cross-Platform Dataset Path Construction
- **Lesson 5:** datetime and os Utilities Supporting Data Workflow Automation

LAB EXERCISES

- Build an Automated File-Path Analyzer for Data Discovery

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

- Import modules safely using custom namespace aliases and targeted component references.
- Extract structural script metadata and environment paths using the sys module.
- Manage cross-platform dataset routing using the modern pathlib framework.
- Apply datetime and os utilities to support data workflow automation.
- Build an automated file-path analyzer for local source data discovery in Google Colab.

MODULE 4 Dependency Management and ML Runtimes

107 min

This module outlines how to construct clean execution environments, install external libraries, and avoid production software conflicts. Students isolate projects using venv, manage packages with pip and Poetry, and configure third-party AI dependencies inside Google Colab workspaces.

LESSONS

- **Lesson 1:** Virtual Environments with venv: Isolating Python Project Architectures
- **Lesson 2:** pip Package Management: Installing, Freezing, and Auditing Dependencies
- **Lesson 3:** Modern Dependency Locking with Poetry and pyproject.toml
- **Lesson 4:** Configuring and Auditing AI Dependencies Inside Google Colab Workspaces

LAB EXERCISES

- Auditing and Pinning ML Dependencies in Google Colab

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

- Isolate project architectures using Python's built-in virtual environment (venv) module.
- Track and maintain enterprise package dependencies using pip and modern lockfile managers like Poetry.
- Configure and audit third-party AI dependencies inside active Google Colab workspaces.
- Generate and interpret requirements.txt and pyproject.toml dependency manifests.
- Initialize a custom machine learning dependency environment for model distribution.

MODULE 5 Algorithmic Branching and Decision Logic

142 min

This module shows how to guide script workflows conditionally based on changing inputs and telemetry. Students construct if-elif-else trees, apply modern structural pattern matching with match-case, and establish data gatekeeping rules to prevent nested block failures.

LESSONS

- **Lesson 1:** if, elif, and else: Building Standard Conditional Decision Trees
- **Lesson 2:** Truthiness, Boolean Operators, and Ternary Expressions
- **Lesson 3:** Structural Pattern Matching with match-case in Python 3.10+
- **Lesson 4:** Guard Clauses and Gatekeeping Rules for Data Frame Safety
- **Lesson 5:** Nested Conditionals vs. Flat Branching: Readability and Maintenance Trade-offs

LAB EXERCISES

- Build a Threshold Routing Matrix with Branching Logic

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

- Construct standard conditional trees with if, elif, and else blocks.
- Execute advanced pattern matching routines using modern structural match-case statements.
- Prevent nested block failures by establishing clear gatekeeping rules for data frames.
- Combine boolean operators and truthiness rules to simplify complex branch conditions.
- Design a threshold routing matrix for automated data evaluation in Google Colab.

MODULE 6 Loops and Iterative Data Pipelines

142 min

This module covers repetitive processing structures, highlighting methods to traverse batch files or array records safely. Students build for and while loops, apply break, continue, and pass controls, and construct protective timeout boundaries.

LESSONS

- **Lesson 1:** for Loops: Iterating Over Sequences, Ranges, and Collections
- **Lesson 2:** enumerate() and zip(): Index-Aware and Parallel Sequence Iteration
- **Lesson 3:** while Loops: Condition-Based Repetition with Protective Timeout Boundaries
- **Lesson 4:** break, continue, and pass: Dynamic Flow Control Inside Loops
- **Lesson 5:** else Clauses on Loops and Nested Loop Patterns

LAB EXERCISES

Building an Iterative Sales Data Pipeline

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

Build explicit sequence iteration routines using the for statement.

Construct conditional while loops with built-in maximum timeout protective boundaries.

Alter looping sequences on-the-fly with break, continue, and defensive pass placeholders.

Apply enumerate() and zip() to iterate over indexed and paired data sequences simultaneously.

Build a file stream monitor that processes records iteratively up to specific limits in Google Colab.

MODULE 7 Pythonic Comprehensions and Performance Optimization

125 min

This module introduces high-efficiency syntactical patterns to streamline memory consumption when transforming large datasets. Students implement list, dictionary, and set comprehensions, stream high-volume data with generators, and write clean output using optimized f-strings.

LESSONS

- **Lesson 1:** List Comprehensions: Single-Line In-Memory Data Transformation
- **Lesson 2:** Dictionary and Set Comprehensions: Inline Mapping and Deduplication
- **Lesson 3:** Generator Expressions: Lazy Evaluation for High-Volume Data Streaming
- **Lesson 4:** f-Strings: Optimized Variable Injection and Numeric Formatting
- **Lesson 5:** Comprehension Readability and PEP 8 Guidelines for Pythonic Code

LAB EXERCISES

Pythonic Comprehensions and Generator Expressions in Google Colab

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

Implement single-line list, dictionary, and set comprehensions for in-memory data conversions.

Stream high-volume datasets sequentially using lazy-loaded generator expressions.

Inject variable text and numerical measurements cleanly using optimized f-strings.

Compare comprehension and generator performance for large-scale ML data transformation tasks.

Refactor data cleaning routines into optimized comprehensions inside Google Colab.

MODULE 8 File System Pipelines and Stream I/O

89 min

This module outlines the mechanisms to manage permanent text records, data dumps, and log profiles safely. Students build console I/O flows, read and write raw data files using open(), and enforce resource cleanup with context managers.

LESSONS

- **Lesson 1:** Console I/O: print() and input() for Interactive Data Collection
- **Lesson 2:** open() and File Modes: Reading and Writing Raw Data Files
- **Lesson 3:** Context Managers: Enforcing Automatic Resource Cleanup with with Statements
- **Lesson 4:** Efficient Line-by-Line Iteration Over Large Text Files

LAB EXERCISES

- Reading, Writing, and Streaming File Data in Python

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

- Build output structures that stream information cleanly through console interactions.
- Read and write raw data files from persistent storage arrays using the open() mechanism.
- Use context managers (with statements) to enforce resource cleanups and secure file closures automatically.
- Apply file modes (r, w, a, r+) appropriately based on data persistence requirements.
- Create a diagnostic script to parse and modify data dumps safely in Google Colab.

MODULE 9 Data Serialization and Cloud API Interactivity

150 min

This module highlights standard networking transmission configurations used to interface with modern external AI web systems. Students convert Python objects to JSON, issue programmatic REST API calls, and stream online parameters using urllib and requests.

LESSONS

- **Lesson 1:** JSON Fundamentals: Structure, Syntax, and Python Compatibility
- **Lesson 2:** The json Module: Serializing and Deserializing Python Objects
- **Lesson 3:** REST API Concepts: HTTP Methods, Status Codes, and Request Headers
- **Lesson 4:** urllib: Native Python HTTP Requests Without External Dependencies
- **Lesson 5:** The requests Library: Simplified REST API Consumption and Response Handling
- **Lesson 6:** API Authentication Patterns: Tokens, Keys, and Secure Header Injection

LAB EXERCISES

Serializing Python Data and Calling REST APIs with JSON

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

Convert Python object trees into cross-compatible JSON layouts and strings.

Handle data interactions with cloud platforms using programmatic REST API calls.

Stream online network parameters utilizing the native urllib and requests frameworks.

Authenticate API requests using header-based tokens and query string parameters.

Pull live web data and transform it into local workspace configurations in Google Colab.

MODULE 10 **Modularizing Code with Functional Programming** 142 min

This module covers procedural abstraction, defining reusable routines, and managing data lifecycle operations cleanly. Students define functions with def, control argument types and defaults, manage variable scope, and apply lambda expressions.

LESSONS

- **Lesson 1:** Defining Functions with def: Structure, Parameters, and Return Values
- **Lesson 2:** Argument Types: Positional, Keyword, Defaults, and Variable-Length Packing
- **Lesson 3:** Variable Scope and the LEGB Rule: Local, Enclosing, Global, and Built-In
- **Lesson 4:** Lambda Expressions and Higher-Order Functions: map() and filter()
- **Lesson 5:** Annotating Functions with Type Hints and Docstrings

LAB EXERCISES

- Building a Modular Data Transformation Library

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

- Construct user-defined functions utilizing the explicit def statement.
- Control function inputs safely using positional, keyword, and fallback default arguments.
- Manage data visibility boundaries using local, nonlocal, global, and returned variables.
- Apply lambda expressions for concise single-expression functional transformations.
- Build a library of modular, testable feature transformation functions in Google Colab.

MODULE 11 Code Standards, PEP 8, and Maintainability

125 min

This module emphasizes styling syntax correctly to ensure collaborative code remains clear and review-ready. Students document code with comments and docstrings, apply PEP 8 guidelines, and automate notebook formatting using Black and Flake8.

LESSONS

- **Lesson 1:** Inline Comments and Docstrings: When and How to Document Python Code
- **Lesson 2:** PEP 8 Layout Rules: Indentation, Line Length, Blank Lines, and Whitespace
- **Lesson 3:** PEP 8 Naming Conventions and Import Organization
- **Lesson 4:** Flake8: Linting Python Code and Interpreting Style Violation Reports
- **Lesson 5:** Black: Opinionated Auto-Formatting for Zero-Debate Codebases

LAB EXERCISES

- Audit and Reformat a Raw Machine Learning Script with PEP 8, Black, and Flake8

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

- Document code files cleanly with inline comments and standard docstrings.
- Apply PEP 8 architectural guidelines to enforce stylistic structure across Python files.
- Automate notebook formatting using linters and format checkers like Black and Flake8.
- Identify and remediate common PEP 8 violations in raw machine learning scripts.
- Format and audit a piece of raw logic into an enterprise-ready module in Google Colab.

MODULE 12 **Debugging and Algorithmic Diagnosis** 71 min

This module focuses on analytical tools used to step through executions, locate logical traps, and track code health. Students differentiate error types, use breakpoint() and interactive notebook debuggers, and inspect variables step-by-step.

LESSONS

- **Lesson 1:** Python Error Taxonomy: Syntax, Logic, and Runtime Failure Modes
- **Lesson 2:** breakpoint() and pdb: Pausing Execution and Stepping Through Code
- **Lesson 3:** Google Colab Interactive Debugger and Variable Inspector
- **Lesson 4:** Common ML Script Bugs: Shape Mismatches, Off-By-One Errors, and None Propagation

LAB EXERCISES

- Debugging and Algorithmic Diagnosis in Google Colab

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

- Differentiate between syntax errors, logical flow traps, and live runtime data failures.
- Pause system execution states at explicit boundaries using the native breakpoint() hook.
- Inspect variable values step-by-step using modern interactive notebook debuggers.
- Interpret Python traceback messages to pinpoint the source file, line number, and error type.
- Trace and repair an unhandled data processing logic flaw within Google Colab.

MODULE 13 Exception Handling and System Logging Workflows

150 min

This module describes how to handle application errors gracefully without crashing running workflows or disrupting systems. Students safeguard unstable operations with try-except blocks, implement fallback logic with else and finally, and record errors using the logging module.

LESSONS

- **Lesson 1:** try and except: Protecting Unstable Operations from Unhandled Crashes
- **Lesson 2:** Catching Multiple Exceptions and Navigating the Python Exception Hierarchy
- **Lesson 3:** else and finally: Conditional Fallback Logic and Guaranteed Cleanup Jobs
- **Lesson 4:** raise and Custom Exception Classes for Domain-Specific ML Error Modeling
- **Lesson 5:** The logging Module: Levels, Handlers, Formatters, and File Output
- **Lesson 6:** Structured Logging Patterns for AI Pipeline Observability

LAB EXERCISES

- Build an Anomaly Detection Logger with Exception Handling

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

- Safeguard unstable runtime actions by separating code inside protective try and except blocks.
- Implement conditional fallback logic routines via else and final data cleanup jobs via finally.
- Define and raise custom exception classes to represent domain-specific ML error conditions.
- Record historical model errors safely utilizing Python's built-in logging module.
- Build an automated system that flags data anomalies into a persistent log registry in Google Colab.

MODULE 14 Machine Learning Integrations and LLM Pipelines

150 min

This module details how to interface with Large Language Models (LLMs) and build production-ready AI pipelines using contemporary techniques. Students inject API credentials securely, construct structured prompts, query live generative AI models, and parse raw response payloads into structured Python objects.

LESSONS

- **Lesson 1:** Securing API Credentials: Colab Secrets, Environment Variables, and python-dotenv
- **Lesson 2:** LLM Client Setup: Authenticating and Configuring the OpenAI API Connection
- **Lesson 3:** Prompt Engineering: Constructing Structured System and User Messages
- **Lesson 4:** Requesting and Parsing JSON-Structured LLM Responses
- **Lesson 5:** Token Parameters, Rate Limits, and Exponential Backoff Retry Logic
- **Lesson 6:** Batch Text Processing Pipelines: Looping Datasets Through LLM API Calls

LAB EXERCISES

Build a Text Categorization Pipeline with the OpenAI API

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

Inject secret API keys securely into runtime notebook spaces using modern workspace credential managers.

Construct structured prompt contexts and query live generative AI models directly inside Python code.

Parse raw string payload definitions back into valid Python schemas, object collections, and tables.

Implement retry logic and rate-limit handling to build resilient LLM API consumers.

Create an end-to-end Python script that leverages an external LLM API to process and categorize raw text datasets.

MODULE 15 Building Python Web Applications and Cloud Deployment

107 min

This module shifts code from text-based notebook environments into web interfaces that users can access globally. Students build interactive data layouts with Streamlit, connect code to GitHub for version control, and map UI controls to active Python execution functions.

LESSONS

- **Lesson 1:** Streamlit Fundamentals: Building Interactive Data Layouts in Pure Python
- **Lesson 2:** Mapping UI Controls to Python Functions: Sliders, Inputs, and File Uploaders
- **Lesson 3:** Git Version Control: Committing Code and Pushing to GitHub
- **Lesson 4:** Deploying to Streamlit Community Cloud and Hugging Face Spaces

LAB EXERCISES

- Build and Deploy a Streamlit Data Dashboard to Streamlit Community Cloud

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

- Build interactive, responsive data layouts using lightweight Python web frameworks such as Streamlit.
- Bridge local application code to public cloud platforms using Git repository version control.
- Map UI controls like sliders, text inputs, and file upload modules directly to active Python execution functions.
- Deploy a Python web application publicly on a free platform via GitHub integration.
- Link a local Python web app project to a GitHub repository and launch it live on a free deployment platform.

MODULE 16 AI-Powered Web Application Capstone

158 min

This optional extension module, delivered at the instructor's discretion, serves as a comprehensive integration of all course milestones. Students architect full-stack AI web applications by combining LLM API calls, data processing pipelines, Streamlit UIs, and GitHub-based cloud deployment into a single production-ready project.

LESSONS

- **Lesson 1:** AI Web Application Architecture: Separating UI, Logic, and API Integration Layers
- **Lesson 2:** Streamlit Session State and Response Caching for Responsive AI Interfaces
- **Lesson 3:** Capstone Option A: Designing the Intelligent Content Summarizer and Sentiment Analyzer
- **Lesson 4:** Capstone Option B: Designing the Predictive Analytics Studio with Tabular Data Upload

LAB EXERCISES

- Build and Deploy an AI-Powered Streamlit Web Application

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO

- Architect clean web layouts that segment backend machine learning model calls from user interface components.
- Optimize API request handling to maintain UI responsiveness during high-compute AI operations.
- Integrate all course skills—data structures, file I/O, API consumption, exception handling, and logging—into a single cohesive AI application.
- Deploy a completed AI-powered web application publicly using GitHub and a free cloud hosting platform.
- Evaluate and present a production-grade AI application demonstrating end-to-end Python proficiency.